

Ambiente virtuale

Ambiente virtuale

Come isolare le dipendenze dei progetti Python con gli ambienti virtuali.

Perché ti serve un ambiente virtuale

Immagina di avere due progetti Python sul tuo computer:

- **Progetto A** funziona con la versione 2.25 della libreria `requests`
- **Progetto B** ha bisogno della versione 2.31 della stessa libreria

Se installi tutto "globalmente" (cioè direttamente sul tuo Python di sistema), i due progetti entrano in conflitto: puoi avere una versione sola, e l'altra non funziona.

Gli **ambienti virtuali** risolvono questo problema: ogni progetto ha la sua "bolla" separata con le proprie librerie, nella versione giusta.

Cos'è un ambiente virtuale?

Un ambiente virtuale è una **cartella isolata** che contiene una copia di Python e di tutte le librerie installate per quel progetto. È separata dal Python del sistema e dagli altri progetti.

Pensa a uno zaino: ogni studente ha il suo, con i propri libri e materiali. Anche se due studenti usano lo stesso manuale, le loro copie sono indipendenti.

Creare un ambiente virtuale

Python include già il modulo `venv` per creare ambienti virtuali. Nella cartella del tuo progetto, esegui:

```
python -m venv venv
```

Questo crea una sottocartella chiamata `venv` con tutto il necessario. Puoi darle qualsiasi nome, ma `venv` o `.venv` sono le convenzioni più usate.

Attivare l'ambiente virtuale

Prima di poterlo usare, devi "attivarlo":

Su macOS e Linux:

```
source venv/bin/activate
```

Su Windows:

```
venv\Scripts\activate
```

Quando l'ambiente è attivo, il nome appare all'inizio del prompt del terminale:

```
(venv) $
```

Da questo momento, qualsiasi `pip install` installerà le librerie **solo in questo ambiente**, non sul sistema globale.

Disattivare l'ambiente virtuale

Quando hai finito di lavorare al progetto:

```
deactivate
```

Il prompt torna alla normalità, senza `(venv)`.

Flusso di lavoro tipico

Ecco la sequenza completa per un nuovo progetto:

```
# 1. Crea la cartella del progetto
mkdir mio-progetto
cd mio-progetto

# 2. Crea l'ambiente virtuale
python -m venv venv

# 3. Attivalo
source venv/bin/activate      # macOS/Linux
# venv\Scripts\activate      # Windows (alternativa)

# 4. Installa le librerie necessarie
pip install requests pandas

# 5. Lavora sul progetto...

# 6. Salva le dipendenze in un file per condividerle
pip freeze > requirements.txt
```

Ripristinare un progetto su un altro computer

Quando qualcuno riceve il tuo progetto (o tu stesso lo riapri su un altro computer):

```
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt  # Installa tutte le librerie elencate nel
file
```

Non aggiungere **venv** a Git

La cartella **venv** è pesante (può occupare centinaia di megabyte) e si può sempre ricreare con **pip install -r requirements.txt**. Non ha senso caricarla su Git.

Aggiungi queste righe al file **.gitignore** del tuo progetto:

```
venv/  
.venv/  
__pycache__/  
*.pyc
```

Strumenti alternativi (per il futuro)

Strumento	Descrizione
venv	Già incluso in Python — ideale per iniziare
virtualenv	Come venv ma con qualche funzione in più
conda	Molto usato in ambito scientifico (Anaconda)
poetry	Gestisce dipendenze e packaging in modo moderno
uv	Strumento moderno e molto veloce

Per ora, **venv** è più che sufficiente.

Verificare dove sei

```
# Su macOS/Linux: mostra quale Python sta usando  
which python  
  
# Su Windows: mostra quale Python sta usando  
where python  
  
# Mostra le librerie installate nell'ambiente attivo  
pip list
```